



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 11

по курсу «Языки и методы программирования»

«Рекурсивный спуск. Парсинг_generic_типов»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Реализация Parser	1
Конструктор и основной метод	2
parseGener()	2
parseIdent()	3
parseTail()	3
parseArgs()	4
parseList()	4
printTree()	5
Тестирование	5
Простой пример	5
Сложный пример	6
Класс Test	7
Вывод	7

Цель работы

Реализовать парсер для `_generic_` типов языка Java (например, `Map[K, V]`, `List[T]`) с использованием метода рекурсивного спуска. Парсер должен разбирать входную строку и строить синтаксическое дерево.

Задачи:

- Изучить метод рекурсивного спуска.
- Реализовать класс `Parser` для разбора `_generic_` типов.
- Обработать вложенные типы и списки аргументов.
- Вывести дерево разбора.

Теоретические сведения

Метод рекурсивного спуска (recursive descent parsing) — это техника парсинга, при которой парсер рекурсивно вызывает себя для разбора различных частей грамматики. Каждый нетерминал грамматики представлен отдельной функцией.

grammar для `_generic_` типов: Грамматика:

- `type = ident (brackets)`
- `ident = имя типа (буквы и цифры)`
- `brackets = [args]`
- `args = type (comma type)...`

Реализация Parser

Внутренний класс `Ident` представляет узел дерева:

Parser.java

```
private static class Ident {  
    String name;  
    Ident left;  
    Ident right;  
}
```

- name — имя типа (Map, List, Int, String)
- left — параметры типа (если есть)
- right — следующий элемент в списке

Конструктор и основной метод

Parser.java

```
private String input;  
private int pos;  
public Ident root;  
private boolean errorOccurred = false;  
  
public Parser(String str) {  
    this.input = str.trim();  
}  
  
public void parse() {  
    root = parseGener();  
    if (errorOccurred) {  
        System.err.println("Syntax error at parsing, position: " +  
pos);  
        System.exit(1);  
    }  
    if (pos < input.length()) {  
        System.err.println("Syntax error: unexpected characters at  
position " + pos);  
        System.exit(1);  
    }  
}
```

parseGener()

Разбор основного типа:

Parser.java

```
private Ident parseGener() {
    if (errorOccurred)
        return null;
    while (pos < input.length() && input.charAt(pos) == ' ') {
        pos++;
    }
    Ident ident = new Ident();
    ident.name = parseIdent();
    if (ident.name.isEmpty() && !errorOccurred) {
        syntaxError();
    }
    if (errorOccurred)
        return null;
    ident.left = parseTail();
    return ident;
}
```

parseIdent()

Разбор имени типа:

Parser.java

```
private String parseIdent() {
    while (pos < input.length() && input.charAt(pos) == ' ') {
        pos++;
    }
    int start = pos;
    while (pos < input.length() && input.charAt(pos) != '['
        && input.charAt(pos) != ']' && input.charAt(pos) != ',') {
        pos++;
    }
    if (pos == start) {
        syntaxError();
        return "";
    }
    return input.substring(start, pos).strip();
}
```

parseTail()

Разбор параметров типа (в квадратных скобках):

Parser.java

```
private Ident parseTail() {
    if (errorOccurred)
        return null;
    while (pos < input.length() && input.charAt(pos) == ' ') {
        pos++;
    }
    if (pos ≥ input.length() || input.charAt(pos) ≠ '[') {
        return null;
    }
    pos++;
    Ident args = parseArgs();
    if (errorOccurred)
        return null;
    while (pos < input.length() && input.charAt(pos) == ' ') {
        pos++;
    }
    if (pos ≥ input.length() || input.charAt(pos) ≠ ']') {
        syntaxError();
        return null;
    }
    pos++;
    return args;
}
```

parseArgs()

Разбор списка аргументов:

Parser.java

```
private Ident parseArgs() {
    if (errorOccurred)
        return null;
    Ident first = parseGener();
    if (first == null) {
        syntaxError();
        return null;
    }
    Ident rest = parseList();
    first.right = rest;
    return first;
}
```

parseList()

Разбор разделённых запятыми аргументов:

Parser.java

```
private Ident parseList() {
    if (errorOccurred)
        return null;
    while (pos < input.length() && input.charAt(pos) == ' ') {
        pos++;
    }
    if (pos < input.length() && input.charAt(pos) == ',') {
        pos++;
        return parseArgs();
    }
    return null;
}
```

printTree()

Вывод дерева разбора:

Parser.java

```
public void printTree() {
    printTree(root, 0);
}

private void printTree(Ident node, int indent) {
    if (node == null)
        return;
    for (int i = 0; i < indent - 1; i++)
        System.out.print("| ");
    if (indent > 0)
        System.out.print("+ ");
    System.out.println(node.name);
    printTree(node.left, indent + 1);
    printTree(node.right, indent);
}
```

Тестирование

Простой пример

simple.txt

```
Map[Int, List[String]]
```

terminal

```
$ java Test < simple.txt
Map
+ Int
+ List
+ String
```

Древовидная структура:

- Map — корень
- Int — параметр типа (левый потомок)
- List — следующий в списке (правый потомок)
- String — параметр List

Сложный пример

complex.txt

```
Map[Map[Map[Map[Int, List[Map[Int, List[String]]]], List[Map[Int,
List[String]]], List[Map[Int, List[String]]], List[Map[Int,
List[String]]]
```

Вывод дерева:

- Map
 1. Map
 1. Map
 1. Int
 2. List
 1. Map
 1. Int
 2. List
 1. String
 2. List
 1. Map
 1. Int
 2. List
 1. String
 2. List
 1. Map
 1. Int
 2. List
 1. String

Класс Test

Чтение из stdin:

Test.java

```
public class Test {
    public static void main(String[] args) throws Exception {
        BufferedReader reader = new BufferedReader(new
InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            sb.append(line).append("\n");
        }

        Parser parser = new Parser(sb.toString());
        parser.parse();
        parser.printTree();
    }
}
```

Вывод

В ходе лабораторной работы был реализован парсер_generic_типов методом рекурсивного спуска:

1. Класс Ident — представление узла дерева
2. parseGener() — разбор основного типа
3. parseIdent() — разбор имени типа
4. parseTail() — разбор параметров в скобках
5. parseArgs() — разбор списка аргументов
6. parseList() — разбор через запятую
7. printTree() — вывод дерева разбора

Парсер корректно обрабатывает простые и вложенные_generic_типы языка Java.